

Análisis y Diseño con Patrones

PRA 1: Principios de diseño, patrones de análisis y arquitectónicos

En esta primera PRA trabajaremos los principios de diseño, patrones de análisis y arquitectónicos en un caso práctico centrado en un sistema de reservas de espacios y recursos en una universidad.

 **NOTA:** Todos los modelos UML que se entreguen deben estar realizados mediante una herramienta de modelado. No se aceptan modelos realizados a mano, ni con herramientas de dibujo tipo *Paint*, ya que estos modelos no tienen rigor. Se recomienda usar la herramienta [VisualParadigm community edition](https://visualparadigm.com/community-edition/).

 **FORMATO:** El documento que entregues con tu propuesta de solución para esta PRA **debe seguir el siguiente formato** (el texto en cursiva corresponde a información que debes rellenar):

Alumno: *...tu nombre...*

Compromiso:

...Certifico que...

Pregunta 1.1:

...tu propuesta de solución...

 **NOTA:** Muchas de las preguntas de esta PRA piden información sobre posibles requisitos del sistema elegido. No se trata de dar respuestas que se ajusten 100% con la realidad ni con la plataforma mencionada, ya que no disponemos de información tan detallada sobre el sistema; sino de dar respuestas plausibles, que tengan sentido desde el punto de vista de lo que se explica en los materiales y que se ajusten a lo que conocemos del sistema. Se valorará especialmente la justificación a las decisiones tomadas, por encima de que se ajusten con exactitud a la realidad.

 **RECUERDA:** Tal como se explica en el Plan docente de la asignatura, esta actividad debe resolverse de forma individual. Por otro lado, no está permitido el uso de herramientas de inteligencia artificial. **En caso de detectar copias o uso de herramientas de IA, se penalizará la actividad con 0 % como nota.**

Para garantizar que has resuelto esta actividad de forma individual y sin uso de herramientas de IA, te pedimos que en tu solución entregada añadas el siguiente compromiso de autorresponsabilidad:

Certifico que he realizado la PRA1 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento "Originalidad en la evaluación" disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

Compromiso de autorresponsabilidad

Escribe aquí el texto con el compromiso de autorresponsabilidad que consta en cursiva en la nota previa. **No escribirlo implica la no corrección de la PRA y una calificación de 0 %.**

Certifico que he realizado la PRA1 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento “Originalidad en la evaluación” disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

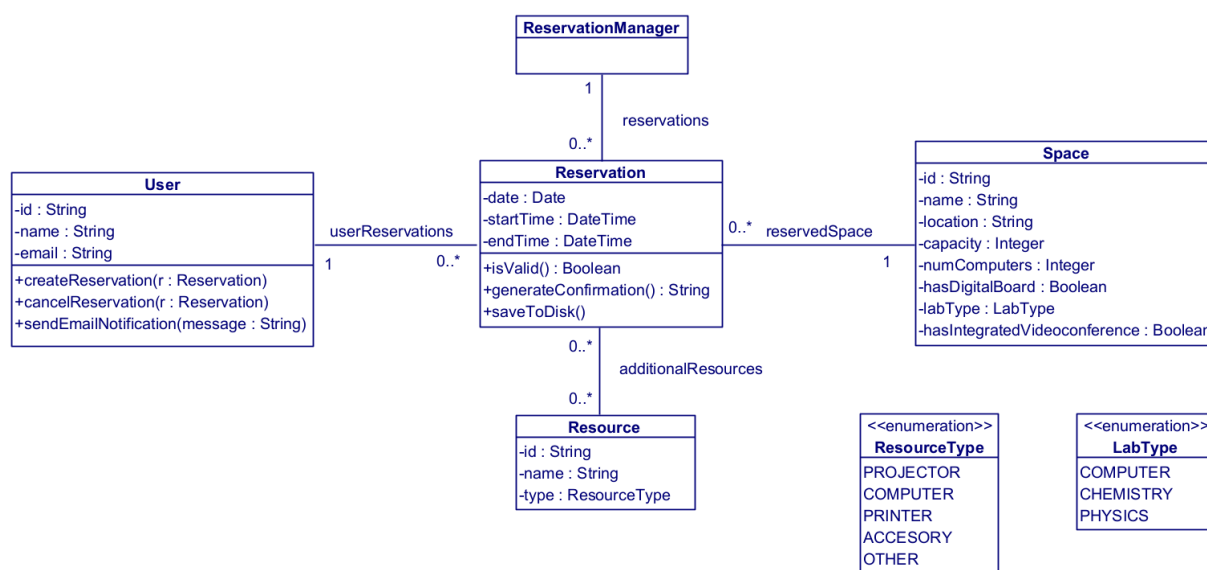
Pregunta 1 (20 %)

Una universidad quiere desarrollar un sistema para gestionar la **reserva de espacios y recursos** del campus. El sistema debe permitir a los usuarios solicitar reservas de distintos tipos de espacios, como aulas, laboratorios o salas de reuniones, así como de ciertos recursos adicionales, como proyectores, cámaras o kits de videoconferencia que se pueden llevar al espacio reservado.

Para los espacios debemos almacenar su identificador, nombre, ubicación y capacidad. Los laboratorios deben incluir también el tipo de laboratorio y el número de ordenadores; las aulas deben indicar si tienen pizarra digital; y las salas de reuniones deben indicar si disponen de videoconferencia integrada.

Cada reserva debe indicar al menos el usuario que la solicita, el espacio principal reservado, el momento previsto de uso y, opcionalmente, otros recursos adicionales asociados.

Se ha propuesto el siguiente diseño inicial para el sistema:



- La clase “Reservation” almacena los datos del usuario, del espacio reservado, de los recursos adicionales, de la fecha y de las horas de inicio y fin.
- La misma clase “Reservation” calcula si la reserva es válida (isValid()), comprueba conflictos con otras reservas y genera el mensaje de confirmación (generateConfirmation()); y guarda la reserva en disco (saveToDisk()).
- La clase “ReservationManager” se encarga de mantener toda la lista de reservas activas, pero delega en la clase “Reservation” la lógica de validación, comprobación de conflictos, generación de mensajes y persistencia.
- La clase “Space” incluye atributos comunes y específicos de todos los tipos de espacios, de modo que una misma clase representa aulas, laboratorios y salas de reuniones.

- La clase “User” mantiene una lista de sus reservas y contiene operaciones para crear y cancelar reservas, así como enviar avisos por correo electrónico.

Restricciones de integridad:

- Los identificadores de espacios y recursos deben ser únicos.
- Las reservas no pueden solaparse en el tiempo para el mismo espacio.
- Los recursos adicionales a una reserva no pueden solaparse con otras reservas.

Además, la universidad quiere que el sistema sea fácil de ampliar, ya que en el futuro podrían incorporarse nuevos tipos de espacios, nuevos recursos y distintos mecanismos de notificación o persistencia.

Pregunta 1.1 (10 %). Analiza críticamente este diseño inicial e indica qué **principios de diseño** del temario no se están respetando. Justifica la respuesta.

Pregunta 1.2 (10 %). Propón un **rediseño** del modelo conceptual y plásmalo mediante un **diagrama de clases UML**.

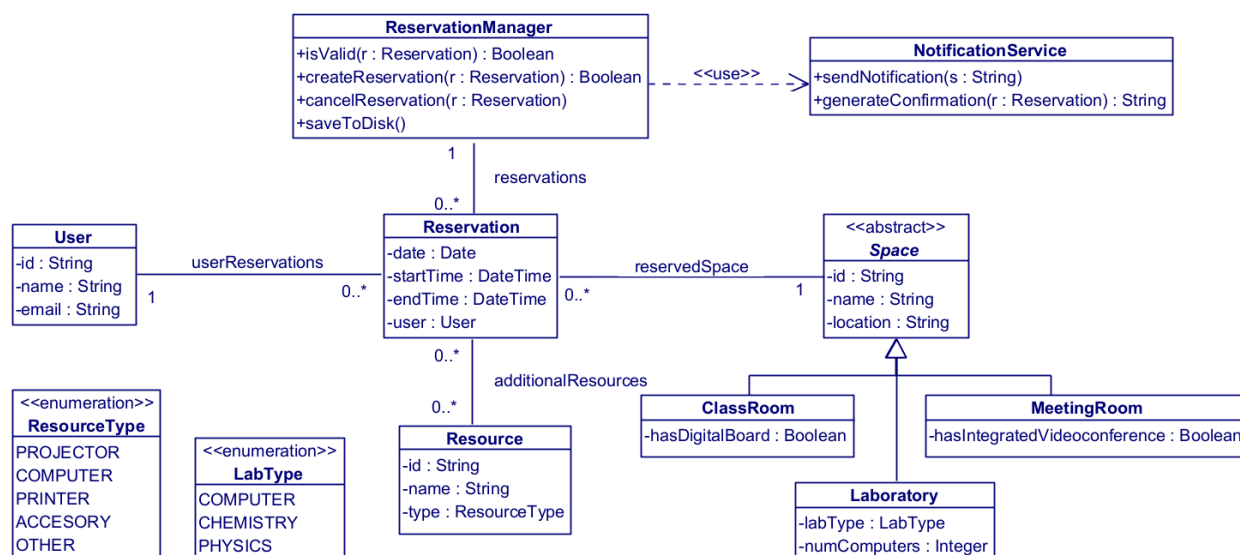
Solución

Pregunta 1.1. El diseño propuesto presenta varios problemas respecto a los principios de diseño del temario:

- **Alta cohesión / violación de SRP:** la clase “Reservation” acumula demasiadas responsabilidades. No solo representa una reserva, sino que también valida reglas, comprueba conflictos, genera mensajes y persiste datos. Estas tareas pertenecen a responsabilidades/motivos de cambio distintos. La clase “Space” también mezcla atributos de distintos tipos de espacios, lo que dificulta su cohesión. Finalmente, la clase “User” también mezcla responsabilidades de gestión de reservas y de notificación, lo que reduce su cohesión.
- **Ley de Demeter:** La clase “User” no tiene una relación directa con “ReservationManager”, pero para crear o cancelar una reserva, probablemente necesite acceder a ella ya que ReservationManager tiene la asociación que almacena todas las reservas, lo que nos lleva a no respetar la ley de Demeter.
- **Open Closed Principle (OCP):** La clase “Space” intenta representar distintos tipos de espacios con atributos que solo se aplican a algunos de ellos. Esto hace que la clase sea difícil de extender sin modificarla, por ejemplo, para añadir un nuevo tipo de espacio con atributos específicos.

En conjunto, el diseño inicial compromete sobre todo **alta cohesión**, **SRP**, **Ley de Demeter** y **OCP** cuando aparezcan nuevos tipos de espacios o servicios externos.

Pregunta 1.2. Una posible mejora consiste en separar claramente responsabilidades del dominio y dependencias técnicas, y especializar adecuadamente los tipos de espacios.



En este rediseño: “Space” es una clase abstracta que define los atributos comunes a todos los espacios, y se especializa en “ClassRoom”, “Laboratory” y “MeetingRoom” para incluir sus atributos específicos. Esto mejora la cohesión y facilita la extensión futura.

“ReservationManager” se encarga de la lógica de validación, creación y cancelación de reservas, delegando en Reservation solo la representación de los datos. Esto mejora la cohesión y respeta el SRP.

Toda la parte de notificaciones se delega a una clase “NotificationService”, lo que mejora la cohesión y facilita la extensión futura para otros tipos de notificaciones.

Pregunta 2 (30 %)

La universidad detecta que realizar una reserva no debe comprobar únicamente solapamientos con otras reservas, sino también validar que esté fuera de periodos de mantenimiento. Estos periodos de mantenimiento se asignan de forma global a toda la universidad para todas las reservas y recursos, por ejemplo, en los periodos intercuatrimestrales.

Por ello, se quiere revisar el modelo temporal del sistema para representar adecuadamente la franja horaria asociada a una reserva y reutilizar ese mismo concepto en otras situaciones.

Podéis asumir la existencia de una clase `MaintenancePeriod` que almacenará una instancia de un periodo de mantenimiento.

MaintenancePeriod
-reason : String

Pregunta 2.1 (5 %). Justifica qué **patrón de análisis** se debe aplicar para agregar esta nueva funcionalidad.

Pregunta 2.2 (10 %). Muestra el **diagrama de clases** con las operaciones necesarias para aplicar este diseño. Para este diseño **no podéis utilizar clases de la librería de Java**, aunque exista una clase aplicable, esperamos que hagáis vuestro propio diseño.

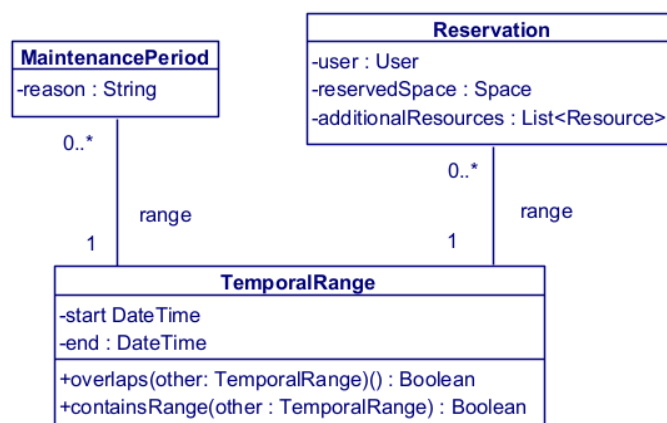
Pregunta 2.3 (15 %). Muestra el **pseudocódigo** de las clases que participan en el diseño, empezando desde la implementación del método que valida una reserva, comprobando las restricciones con otras reservas y con los periodos de mantenimiento.

Solución

Pregunta 2.1. El patrón de análisis más adecuado es **Range**. La idea central es que la información temporal de una reserva no debe quedar reducida a varios atributos primitivos dispersos, sino que conviene modelarla como un concepto del dominio con semántica propia. En este caso, un periodo de mantenimiento o una ventana permitida de reserva son todos ejemplos de **intervalos** sobre los que se necesitan operaciones específicas, como comprobar solapamientos y reutilizar la misma lógica en distintos contextos.

Por tanto, en lugar de repartir esa lógica entre “Reservation”, “Space” u otras clases, conviene introducir una abstracción que represente explícitamente el intervalo temporal.

Pregunta 2.2. El diagrama de clases, una vez añadido el patrón Rango, sería el mostrado en la siguiente figura. Se incluyen solamente las clases involucradas en el patrón.



Esta solución tiene varias ventajas:

- concentra en “TemporalRange” las operaciones del dominio temporal,
- evita duplicar lógica de intervalos en varias clases,
- facilita reutilizar la misma abstracción en distintos casos del sistema,
- y mejora la claridad conceptual del modelo.

Pregunta 2.3.

```

public class Reservation {
    private TemporalRange range;
    private User user;
    private Space space;
    private List<Resource> additionalResources;

    public TemporalRange getRange() {
        return range;
    }
}

public class MaintenancePeriod {

```



```
private TemporalRange range;
private String reason;

public TemporalRange getRange() {
    return range;
}

}

public class TemporalRange {
    private DateTime start;
    private DateTime end;

    public TemporalRange(Datetime start, Datetime end){
        this.start = start;
        this.end = end;
    }

    public DateTime getStart() {
        return this.start;
    }

    public DateTime getEnd() {
        return this.end;
    }

    public boolean overlaps(TemporalRange other) {
        return this.getStart() < other.getEnd() && this.getEnd() > other.getStart();
    }
}

public class ReservationManager {
    private List<Reservation> reservations;
    private OpeningHours openingHours;
    private List<MaintenancePeriod> maintenancePeriods;

    public boolean isValid(Reservation r) {
        // Comprobar solapamientos con otras reservas
        for (Reservation existing : reservations) {
            if (r.range.overlaps(existing.range)) {
                return false;
            }
        }

        // Comprobar que no coincide con periodos de mantenimiento
        for (MaintenancePeriod mp : maintenancePeriods) {
            if (r.range.overlaps(mp.range)) {
                return false;
            }
        }

        return true;
    }
}
```

Pregunta 3 (30 %)

El sistema de reservas actual solo permite reservar recursos individuales, como un proyector o una cámara. Sin embargo, la universidad quiere ampliar el sistema para que también se puedan reservar conjuntos de recursos. Por ejemplo, un equipo de proyección puede incluir un proyector, un portátil y un cable VGA, o un kit para docencia híbrida puede incluir una cámara, un micrófono y un sistema de audio;

El sistema debe permitir trabajar con todos estos elementos de manera uniforme, de forma que sea posible: consultarlos, añadirlos a una reserva, y obtener información agregada, como el número total de recursos incluidos.

Pregunta 3.1 (5 %). Justifica qué **patrón de análisis** se adapta mejor a esta nueva necesidad.

Pregunta 3.2 (10 %). Muestra el **diagrama de clases** con las operaciones necesarias para implementar este diseño, así como sus restricciones de integridad.

Pregunta 3.3 (15 %). Muestra el **pseudocódigo completo** de las clases que participan en la operación `Rreservation::numberOfElements()` que calcula el número total de recursos incluidos en la reserva.

Solución

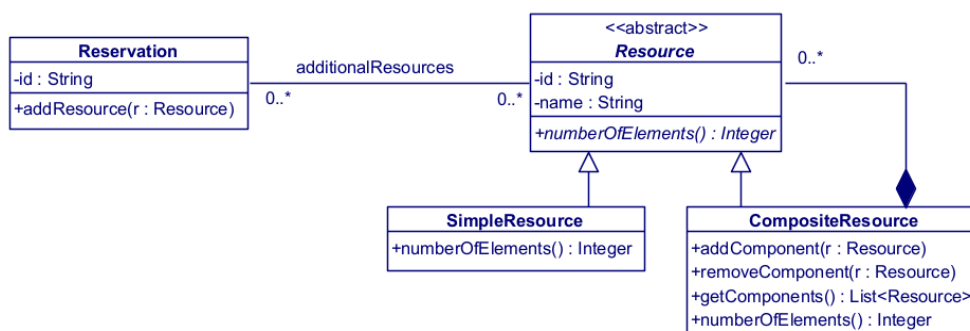
Pregunta 3.1. El patrón de análisis que mejor se adapta a esta necesidad es **Composite**.

La universidad quiere tratar de forma uniforme recursos simples y recursos compuestos. Un proyector o una cámara son recursos simples, mientras que un “Equipo de proyección” o un “Kit para docencia híbrida” son recursos compuestos por otros recursos. Además, el enunciado indica que un recurso compuesto puede incluir otros recursos compuestos, lo que sugiere una estructura recursiva parte-todo.

El patrón Composite permite precisamente:

- definir una abstracción común para todos los recursos,
- representar elementos hoja y compuestos,
- y aplicar operaciones uniformes sobre ambos tipos.

Pregunta 3.2. El diagrama de clases del patrón *Composite* aplicado a este problema se vería así:



Restricciones de integridad:

- Los identificadores de reservas y recursos deben ser únicos.

Pregunta 3.3. Una posible implementación en pseudocódigo sería la siguiente:

```

public class Reservation {
    private List<Resource> resources;

    public Integer numberOfElements() {
        int total = 0;
        for (Resource r : resources) {
            total += r.numberOfElements();
        }
        return total;
    }
}

```

```
public abstract class Resource {
    private String id;
    private String name;

    public abstract Integer numberOfElements();
}

public class SimpleResource extends Resource {
    public int numberOfElements() {
        return 1; // un recurso simple cuenta como uno
    }
}

public class CompositeResource extends Resource {
    private List<Resource> components;

    public void addComponent(Resource r) {
        components.add(r);
    }

    public void removeComponent(Resource r) {
        components.remove(r);
    }

    public List<Resource> getComponents() {
        return components;
    }

    public Integer numberOfElements() {
        int total = 0;
        for (Resource r : components) {
            total += r.numberOfElements(); // delega a cada componente
        }
        return total;
    }
}
```

Pregunta 4 (20 %)

La universidad prevé que, en futuras versiones del sistema, la gestión de reservas deba poder trabajar con distintos servicios externos, por ejemplo:

- Diferentes mecanismos de notificación al usuario, como correo electrónico o mensajería interna;
- o distintos sistemas de persistencia de datos;

La dirección técnica ha decidido que las clases responsables de gestionar las reservas **no deben crear directamente** estos servicios concretos, ya que se quiere poder sustituir una implementación por otra sin modificar la lógica principal del sistema.

Por ejemplo, la misma lógica de creación o cancelación de reservas debería poder funcionar con un notificador por correo electrónico o con un notificador de mensajería interna. O la base de datos podría ser PostgreSQL o MySQL, sin que eso afecte a la clase que gestiona las reservas.

Pregunta 4.1 (5 %). Justifica qué **patrón arquitectónico** se adecúa mejor a esta necesidad.

Pregunta 4.2 (15 %). Muestra el **diagrama de clases** con las operaciones necesarias para implementar este diseño en el que se vea claramente cómo una clase encargada de gestionar reservas recibe desde fuera las dependencias que necesita, sin instanciar implementaciones concretas.

Solución

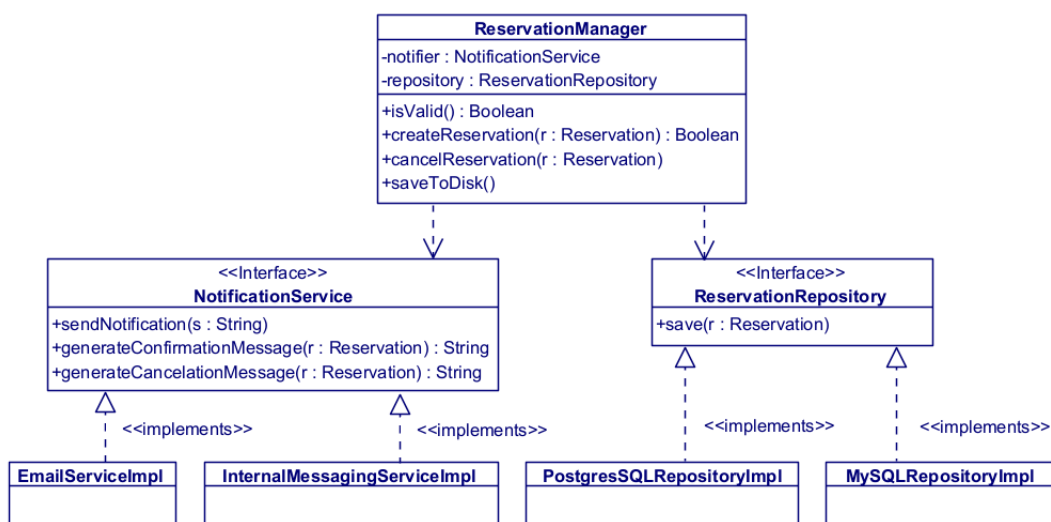
Pregunta 4.1. El patrón arquitectónico adecuado es la **inyección de dependencias**.

La necesidad descrita consiste en evitar que la clase que gestiona las reservas cree directamente objetos concretos para notificar. Si esa clase instanciara por sí misma un notificador por correo, quedaría fuertemente acoplada a esas implementaciones y cualquier cambio obligaría a modificar su código.

La inyección de dependencias permite que la clase declare qué colaboraciones necesita mediante **abstracciones**, y que esas dependencias se le proporcionen desde fuera. Esto facilita:

- sustituir implementaciones sin cambiar la lógica de negocio,
- reducir acoplamiento,
- hacer el sistema más fácil de probar,
- y mejorar la mantenibilidad.

Pregunta 4.2. El diagrama de clases que corresponde a este diseño sería:



En esta solución, “ReservationManager” depende solo de las abstracciones “NotificationService” y “ReservationRepository”. Las implementaciones concretas se crean fuera de la clase y se le pasan, por ejemplo, a través del constructor. Las abstracciones se modelan como interfaces y las implementaciones concretas (EmailService, InternalMessagingService...) implementan estas interfaces.

Así, la misma lógica de gestión de reservas puede funcionar con distintos notificadores, distintos repositorios o distintos servicios de disponibilidad sin modificar “ReservationManager”. Eso es precisamente lo que se busca con la **inyección de dependencias**.

Criterios de evaluación

La nota final de la PEC se calculará a partir de las notas obtenidas en cada una de las preguntas individuales, con el peso indicado en el enunciado. En concreto, este es el criterio:

- Pregunta 1: 20 %
- Pregunta 2: 30 %
- Pregunta 3: 30 %
- Pregunta 4: 20 %

Esta PRA corresponde a un **50% de la nota de Prácticas de la asignatura**.



RECUERDA: Para acogerte a la evaluación continua de la asignatura, deberás entregar todas las PECs y PRAs, y **superar cada una de ellas con al menos 3 puntos**.

Tal como se explica en el plan docente de la asignatura, esta actividad debe resolverse de forma individual. Por otro lado, no está permitido el uso de herramientas de inteligencia artificial. Recuerda que debes incluir el texto indicado en la sección “Compromiso de autorresponsabilidad” de tu PRA. En caso de detectar copias o uso de herramientas de IA, se penalizará la actividad con un 0 % como nota.

Formato de entrega

- Deberás entregar un **único documento** (formato pdf o docx) con tu propuesta de solución para las preguntas del enunciado.
- Recuerda que debes producir diagramas **conforme a la especificación UML** y **ser consistente** en el estilo de diagramas que entregues en todos los ejercicios.
- Todos los documentos se tienen que entregar en el aula virtual antes de las 23:59 horas del **21 de abril de 2026**.
- **No se aceptarán entregas fuera de plazo.**

Propiedad intelectual y plagio

La Normativa académica de la UOC dispone que el proceso de evaluación se cimenta en el trabajo personal del estudiante y presupone la autenticidad de la autoría y la originalidad de los ejercicios realizados.

La ausencia de originalidad en la autoría o el mal uso de las condiciones en las que se realiza la evaluación de la asignatura, es una infracción que puede tener consecuencias académicas graves.

Te recomendamos leer esta [guía sobre el plagio académico y como evitarlo](#). Recuerda que siempre que reutilices contenido de terceros, debes citar la fuente. Puedes leer también esta [guía para saber cómo citar correctamente](#).

El estudiante será calificado con un suspenso (0 %) si se detecta falta de originalidad en la autoría, sea porque haya utilizado material o dispositivos no autorizados, sea porque ha copiado textualmente de internet, o ha copiado apuntes, de otras PECs, de materiales, manuales o artículos (sin la cita correspondiente) o de otro estudiante, o por cualquier otra conducta irregular.